Object Lens

System design and software quality

Private technical companion
Edition 2.1
3 November 2026

Object Lens is a small local browser application for exploring object detection in a photograph. The user can choose a photo, inspect the model's predictions, adjust the minimum displayed score and save an annotated image. A sample-photo action provides a quick way to try the same workflow.

Source: docs/DESIGN.md

This companion explains what each part does, why it exists and how the application is checked. It is written as a learning reference for a validation engineer moving into software automation. The explanations follow one image through the application and one requirement through its automated test.

This edition refreshes the current application after the loading follow-up and the evidence needed for a bounded local release. It explains stage-specific recovery, a loading deadline that tolerates slow but steady connections, visible-output assertions and box-position regression. The exact tested source is identified by file hashes; public website upload remains parked. Passing these checks does not establish general object-recognition accuracy.

Document control	Value
Prepared for	Tan Le Jie
Application source	17f2b9a plus source hashes in the evidence record
Evidence version	docs/evidence/loading-followup-verification.json
Scope	Local Windows and Chrome learning and demonstration workflow

Contents

Scope and user workflow **3**
Objectives and completion 4
Technology and runtime architecture **4**
Server and worker boundaries 6
Configuration and loading feedback 6
Image processing and result behavior **6**
From file bytes to predictions 6
Current image ownership 8
Bounded recovery 8
Reject invalid model output 8
Filtering, samples and PNG output **9**
Minimum model score 9
Try sample 9
Save result 9
Testing with Python, pytest and Selenium **11**
The two commands 11
Controlled checks and actual inference 11
A worked requirement 12
Requirements and test design **13**
Choosing useful cases 14
Visible results and box regression 15
Failure investigation **15**
A real narrow viewport defect 15
A deliberate stale-result fault 16
A prediction mistake is a different kind of finding 16
An unexpected file in a download check 16
A slow connection that could never finish 17
Current recovery wording and assertion gaps 17
Verification and release assessment **17**
Release decision 19
Running and maintaining the project **19**
Use the application 19
Run checks 19
Make a bounded change 20
AI assistance and honest demonstration 20
Terms and source references **21**

Reading this companion

Start with scope and architecture, then follow image processing and the separate testing path. Verification records the current result and its limits. Figures are explanatory diagrams; screenshots are actual captures. Controlled failures, deliberate source faults and real model observations are identified separately.

Scope and user workflow

The intended user is a developer, quality engineer or learner who wants to try a pretrained detector and understand its predictions. Object Lens focuses on one photo at a time, one model and Chrome on Windows. Its small scope keeps the application and its automation understandable.

The source repository remains private. A static public package is prepared locally from an explicit allowlist, but website upload is parked and there is no verified public URL. A local server, source publication and deployment are separate operations.

The main workflow is to choose an image or try the sample, wait for inference, inspect the boxes and matching result rows, adjust the minimum score and save the visible annotations as PNG. Clear removes the current image and its results. Replacing a photo starts a new image workflow.

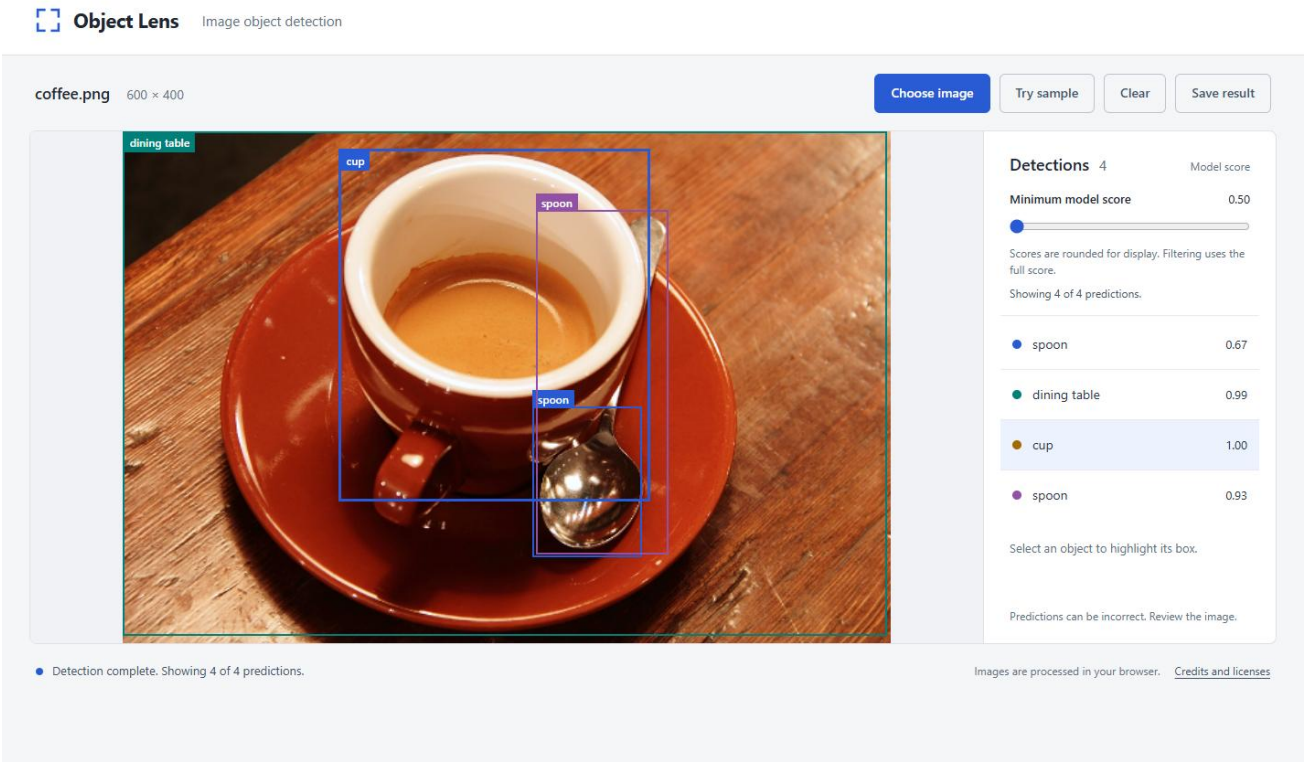


Figure 1 Current application after real inference on the public coffee sample, captured during the 3 November loading follow-up. The selected prediction links its visible row to its box; credits remain available.

Table 1 Product boundaries

Area	Included	Limit
Input	One decodable JPEG or PNG	At most 10 MiB and 16,000,000 pixels
Detection	One pretrained DETR model in the browser	Model classes and recognition limitations remain those of the chosen model
Interaction	Prediction selection, score filter, sample action, replace and Clear	No saved image history or batch workflow
Output	PNG with the current visible boxes, labels and scores	No report strip, cloud upload or accuracy certification
Execution	Local Windows application and headless Chrome tests	No hosted deployment, iOS or Safari qualification

Accounts, databases, camera/video processing, model training, a model picker and additional AI services are outside this version. The purpose is to explore a working AI-enabled interface and software test automation without maintaining a larger service.

Objectives and completion

The first objective is a usable path from an image to inspectable predictions. The second is to translate observable requirements into acceptance criteria and useful automated checks. The third is to retain enough evidence to investigate failures and make a limited, reasoned release decision.

Completion requires the agreed features to work together, relevant tests and UI checks to pass, an actual PNG export to match the visible predictions, and this document to describe the final source. Source publication and online deployment remain separate decisions.

Back to contents

Technology and runtime architecture

The web application and its automated tests are two different programs. Opening the application lets a person choose and inspect photos. Running pytest asks Python to execute checks against isolated application instances. The detector belongs to the application path; pytest and Selenium belong to the checking path.

Table 2 Technology responsibilities

Component	What it does	Why it is used here
HTML and CSS	Define the page, controls and presentation	A single page can use native browser features without a frontend framework
JavaScript	Validates input and coordinates image, prediction, filter and export state	The interface and model runtime execute in the browser
Web Worker	Runs model loading and inference away from the page's main thread	The page can respond to controls while model work is running
Transformers.js	Loads the selected model and calls its object-detection pipeline	Supplies a mature pretrained-model interface
ONNX Runtime Web	Executes the prepared model through CPU WebAssembly	Provides local inference in the target browser
Python HTTP server	Serves the application and prepared assets on 127.0.0.1	A small standard-library server is sufficient for local use
Static website package	Contains the same app, normal-mode configuration, model assets, sample and credits	Allows ordinary static serving without the custom Python routes
Python and pytest	Define checks, prepare fixtures and report assertions	Keep the core automation in one language with a mature test runner
Selenium	Operates an isolated Chrome session and reads its page	Exercises the user-visible browser behavior



Figure 2 Runtime responsibilities. The local server provides files; the browser performs inference and draws PNG output. The separate automated-testing workflow appears in Figure 5. This figure illustrates local Python serving; the static package supplies the same browser files through an ordinary static server.

The prepared application uses local runtime and model files. The worker sets remote-model access off and requests the pinned local model. An initial setup still needs downloads for dependencies, model assets, public samples and potentially the matching Chrome driver. Local inference therefore means that the prepared inference path does not call a hosted prediction service.

The recorded base uses Transformers.js 2.17.2 and ONNX Runtime Web 1.14.0 with a quantized Xenova/detr-resnet-50 model. Exact sources, model revision, file sizes and SHA-256 hashes are recorded in asset-lock.json. The version pins make the tested setup reproducible; they do not mean that these are the newest releases.

Server and worker boundaries

The Python server binds to 127.0.0.1, which addresses this computer. It is not an online demo service. The browser worker owns model loading and inference. The page owns the chosen image, the current request identity and everything displayed to the user.

Running inference in a worker keeps model execution out of the main page thread, but it does not make the computation free or provide a speed guarantee. This version uses CPU/WASM with one thread. It makes no GPU, NPU or semiconductor validation claim.

Configuration and loading feedback

The page first loads runtime configuration under a 15-second deadline covering response headers and the JSON body. The testMode value must be a boolean before a worker can start. A failed request exposes retry and retains a valid selected image. A new valid selection can also retry configuration.

Source: dist/app.js; tests/test_startup.py

Model feedback separates transfer progress from runtime initialization. Received bytes are shown only when a finite total is known. A transfer at 100 percent is not readiness; only the worker ready event establishes that. Worker callbacks must still belong to the current worker instance. Background startup cannot erase an input error or say the page is idle while a file is still being read.

Source: dist/app.js; dist/worker.js; tests/test_loading.py

The static package must be served at the site root because worker asset paths begin with /assets. Browser caching may retain model/runtime downloads; the app keeps selected photo data and current predictions in its page/worker and provides no persistent photo-history feature. Clear resets current image state, not cached model assets.

Source: dist/worker.js; dist/app.js; docs/PUBLIC_DEMO.md

Back to contents

Image processing and result behavior

From file bytes to predictions

When a person chooses a file, the application checks the allowed format, file size, successful decoding and image dimensions. A filename extension alone is insufficient: a file called photo.png can contain unreadable or unsupported bytes. Validation prevents unsuitable input from reaching the expensive model call.

The real worker passes the image to the object-detection pipeline. The model uses learned parameters from its earlier training to compute predictions from the image pixels. The application does not select a predefined answer from the filename, and uploading an image does not train or update the model.

Excerpt 1 Real inference call from the final worker

```
const predictions = await detector(data.image, { threshold: 0.5, percentage: true });
```

Source: dist/worker.js: infer

Table 3 One prediction

Field	Meaning	How the interface uses it
label	A supported class name predicted by the model	Text in the result row and box label
score	The model's score for that prediction	Displayed beside the label and compared with the display minimum
box	Normalized xmin, ymin, xmax and ymax coordinates	Position and size of the box relative to the decoded image
Stable prediction identity	An application identity retained for the current result set	Keeps selection and color associated with the same prediction when filtering

A score is not a measured accuracy percentage. Two predictions can describe the same visible object, and a high-scoring prediction can still be incorrect. The interface therefore counts predictions rather than claiming a verified count of physical objects.

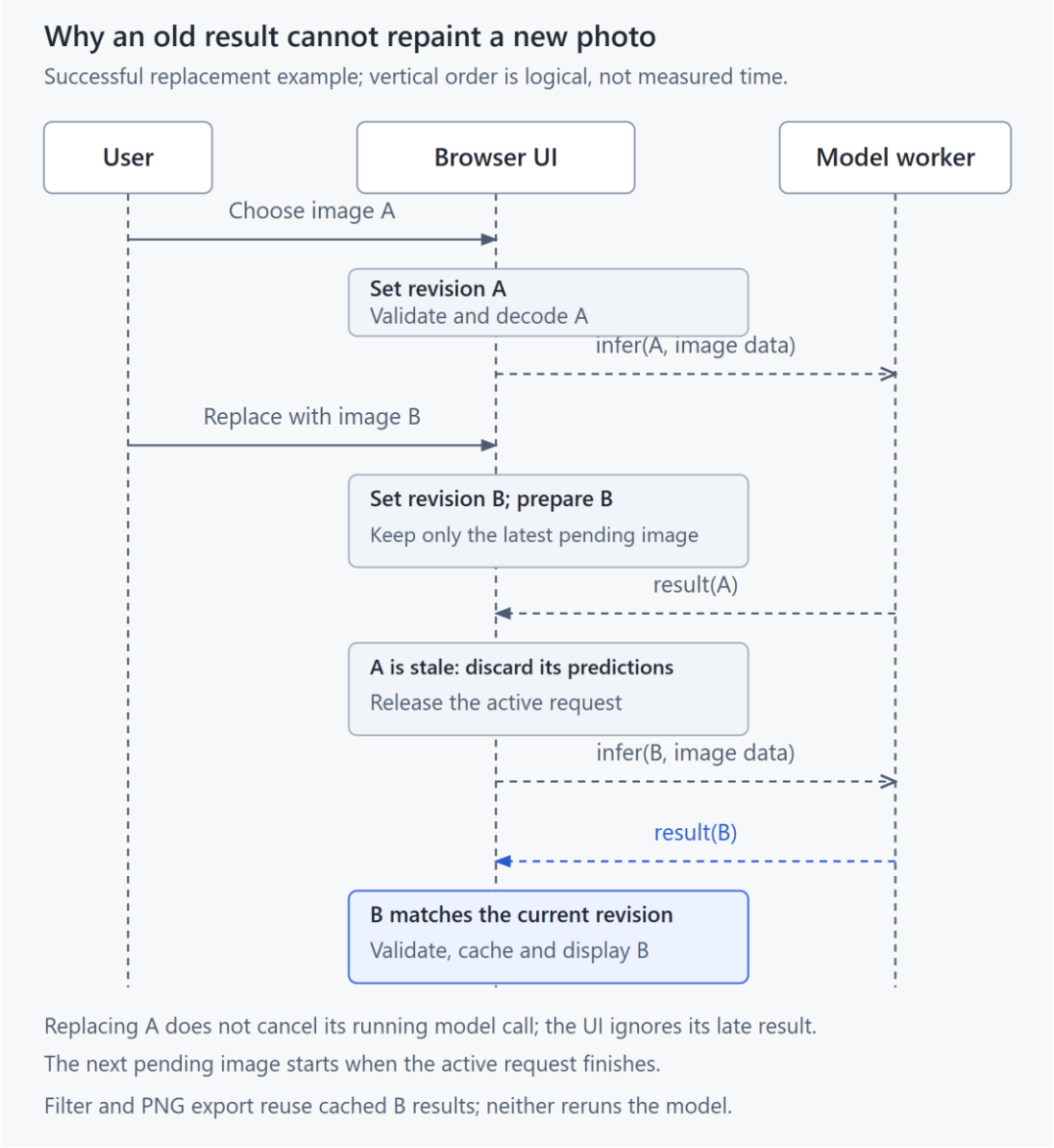


Figure 3 Image A finishes after the user selects image B. The application discards A's stale reply, then processes the latest pending image. Filtering and PNG export use retained results after inference.

Current image ownership

Every new image workflow has a revision identity. A result is accepted only when its request belongs to the current image. If an earlier image finishes after replacement or Clear, its reply cannot repaint the new page. This is an important distinction: discarding an old reply does not promise that the CPU work was interrupted.

The worker handles one active request, while the page retains at most the latest pending image. This prevents rapid replacements from turning into an unbounded list of obsolete requests.

Bounded recovery

Model transfer fails only after 45 seconds without a loading event, so a slow but steady connection can finish the 43 MB download. Once the model file finishes, runtime initialization has a fixed 120-second deadline, and inference has 90 seconds. The failed worker is terminated. A stalled transfer asks the user to check the connection, an initialization timeout says the model took too long to start, and an inference timeout describes image analysis. Clear removes the photo and replaces the earlier explanation with a generic detector-unavailable message offering restart. It does not claim readiness or reclassify the prior failure as startup failure. Retry or a new valid image recreates the worker.

Source: dist/app.js; tests/test_recovery.py

Normal controlled-worker loading retains the production deadlines. Dedicated loading tests accelerate only the 45-second stall and 120-second initialization timers in their isolated browser, and check a stalled transfer, a slow steady transfer that must not time out, and an initialization hang. Controlled inference uses a four-second deadline. A simulated failure proves that recovery works under the chosen condition, not that the real model was observed hanging.

An empty prediction set, an invalid image and an inference failure have different meanings. Empty results mean the detector returned no predictions under its configured output threshold. Invalid input failed validation. An inference failure means the requested processing did not complete successfully. The page should communicate the correct state and offer the applicable recovery.

Reject invalid model output

The page validates the returned array and each prediction before displaying results. Labels must be nonblank strings, scores must be finite numeric values from 0 to 1, and the four coordinates must be finite numbers. Coordinates are clamped to the image bounds only if the resulting rectangle has positive area. Missing fields, invalid scores and reversed, collapsed or wholly out-of-bounds rectangles are rejected.

Source: dist/app.js; tests/test_recovery.py

A malformed prediction invalidates the response as a whole, including a batch with an otherwise valid entry. The page reports the problem, leaves no misleading boxes or result rows, and allows a later valid response to recover. The controlled fixtures intentionally create these failures; the real-model observations did not produce them.

Back to contents

Filtering, samples and PNG output

Minimum model score

The display filter ranges from 0.50 to 1.00 in steps of 0.01. It starts at 0.50 for a newly chosen or sample image and resets on Clear. The filter compares the unrounded model score with the selected minimum and includes a prediction when its score is greater than or equal to that value.

The real detector is still requested with its existing 0.5 output threshold. Raising the display minimum hides predictions already returned by that call; lowering it restores retained predictions down to the available 0.50 floor. The filter cannot recover predictions the detector did not return, and it does not change the model's parameters.

In the recorded base coffee run, one spoon prediction scored about 0.67 and another about 0.93. A display minimum of 0.80 would hide the first and retain the second. This is a concrete filtering example, not evidence that the retained prediction is necessarily correct.

The boxes, list and visible count update from the same filtered set. Predictions retain their original identities and colors. If the selected prediction remains visible, selection is preserved. If it is hidden, the first remaining prediction becomes selected; if none remains, selection is cleared.

Source: dist/app.js; tests/test_filter.py

When the model returned predictions but the filter hides all of them, the page says that no predictions meet the minimum score and invites the user to lower it. The image stays visible and the slider stays enabled. If the detector returned an empty set, the page uses a different empty message. Displayed scores are rounded to two decimals; comparisons use the original score.

Source: dist/app.js; tests/test_filter.py

Try sample

Try sample loads the existing public coffee photo through the ordinary input and real-inference flow. Its purpose is to make the application easy to try without browsing through folders. The photo is a genuine input, not a shortcut to a fixed prediction list.

The sample file and its attribution already belong to the prepared local assets. A missing or unreadable sample produces an actionable error and a new Try sample attempt can recover. A 15-second deadline aborts a fetch that does not finish.

Source: dist/app.js; tests/test_sample.py

Sample loading reserves a new image revision before starting its fetch. Clear or a newer image selection aborts that request and retires its identity. A late response cannot become the current image. Controlled delayed-response tests exercise both Clear and replacement; this is verified prevention of a design risk, not a claim that a production incident occurred.

Source: dist/app.js; tests/test_sample.py

Save result

Save result creates a PNG of the decoded, correctly oriented image at its original pixel resolution, with only the currently visible prediction boxes, labels and model scores. It includes no summary strip. The source image file is not overwritten.

The on-screen boxes are HTML elements over the image. PNG export draws the same visible prediction data onto a canvas in the page's main thread. Coordinates are scaled to the decoded image

dimensions, and text labels include the class and rounded model score. Label backgrounds and text are kept within the canvas. The saved PNG is therefore checked separately from the screen.

Source: dist/png-export.js; tests/test_export.py

Export captures one image, one visible prediction set and the selected minimum. Clear, replacement or a filter change invalidates an export still being encoded, so it cannot initiate a stale download. Save is disabled when no predictions are visible and while an export is being prepared. An encoder failure or the 15-second encoding deadline leaves the displayed result intact and makes Save available for another attempt.

Source: dist/app.js; dist/png-export.js; tests/test_export.py

The application reports PNG download started after handing the generated file to the browser. It does not claim that the operating system has confirmed a completed save. The filename includes the source image stem and the display minimum, for example coffee-object-lens-min-0.80.png.

Source: dist/app.js; dist/png-export.js

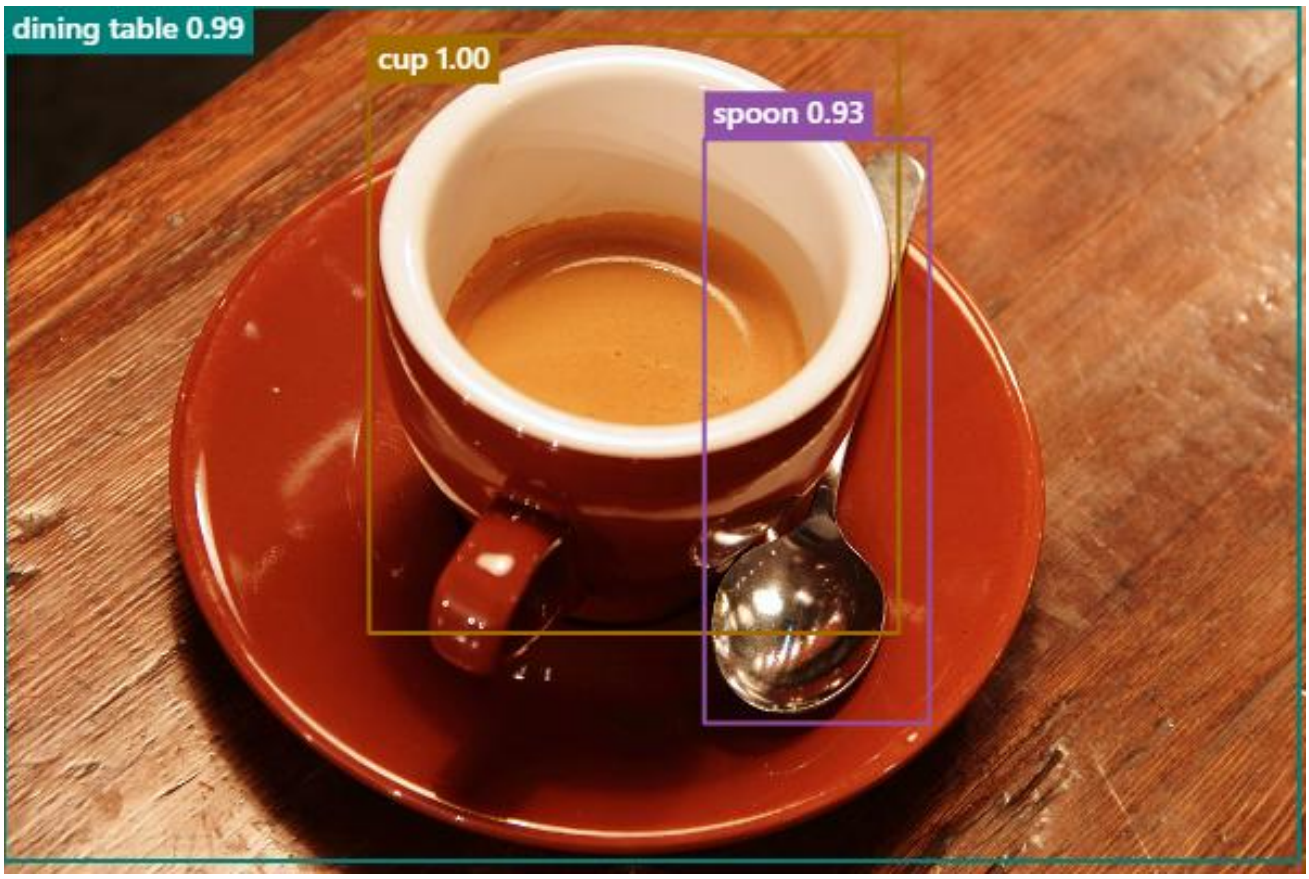


Figure 4 Actual PNG saved during the current loading follow-up from the coffee sample at minimum score 0.80. It contains the photo and three visible prediction annotations, without application controls.

The saved image contains model predictions. A recipient should still review the photo and interpret the labels as predictions rather than verified facts.

Back to contents

Testing with Python, pytest and Selenium

The two commands

Table 4 Application and test commands

Command from the project folder	What it does	Why to use it
.\.venv\Scripts\python.exe serve.py	Starts the local application server	Use or demonstrate the application manually
.\.venv\Scripts\python.exe -m pytest	Runs the written automated tests	Check whether application behavior matches the expected rules

The server command stays running until it is stopped. The pytest command finishes after the test functions and their cleanup. Tests start their own temporary servers and headless Chrome sessions, so the manually started app is not a prerequisite.

Python supplies the language in which the tests are written. pytest discovers test files, arranges fixtures, executes test functions and reports outcomes. Selenium follows the actions described by a test: selecting a file, clicking a control and reading the displayed page. Assertions compare the observed values with the expected rule.

Controlled checks and actual inference

The controlled test worker deliberately supplies known predictions, delays and failures. This lets a test reliably exercise an empty set, a timeout or a stale response. It does not establish how accurately the real model recognizes objects. Normal application mode uses the real worker; controlled mode must be enabled explicitly and is visibly labelled.

Tests marked inference run the actual prepared model. Three photo cases assert person, cup and cat text/scores, compare the selected coordinates with earlier pinned-model outputs and measure actual rendered geometry. A fourth case serves the generated package through an ordinary static server and exercises actual inference, filtering and PNG download. These are limited integration/regression checks, not an accuracy benchmark.

Run a focused group

```
# Non-model application, server, package and harness checks
.\.venv\Scripts\python.exe -m pytest -m "not inference"

# Actual-model checks
.\.venv\Scripts\python.exe -m pytest -m inference
```

In the second command, Python interprets the first -m as run this installed module. pytest interprets the later -m as select tests with this marker. The identical spelling belongs to two different command-line interfaces.

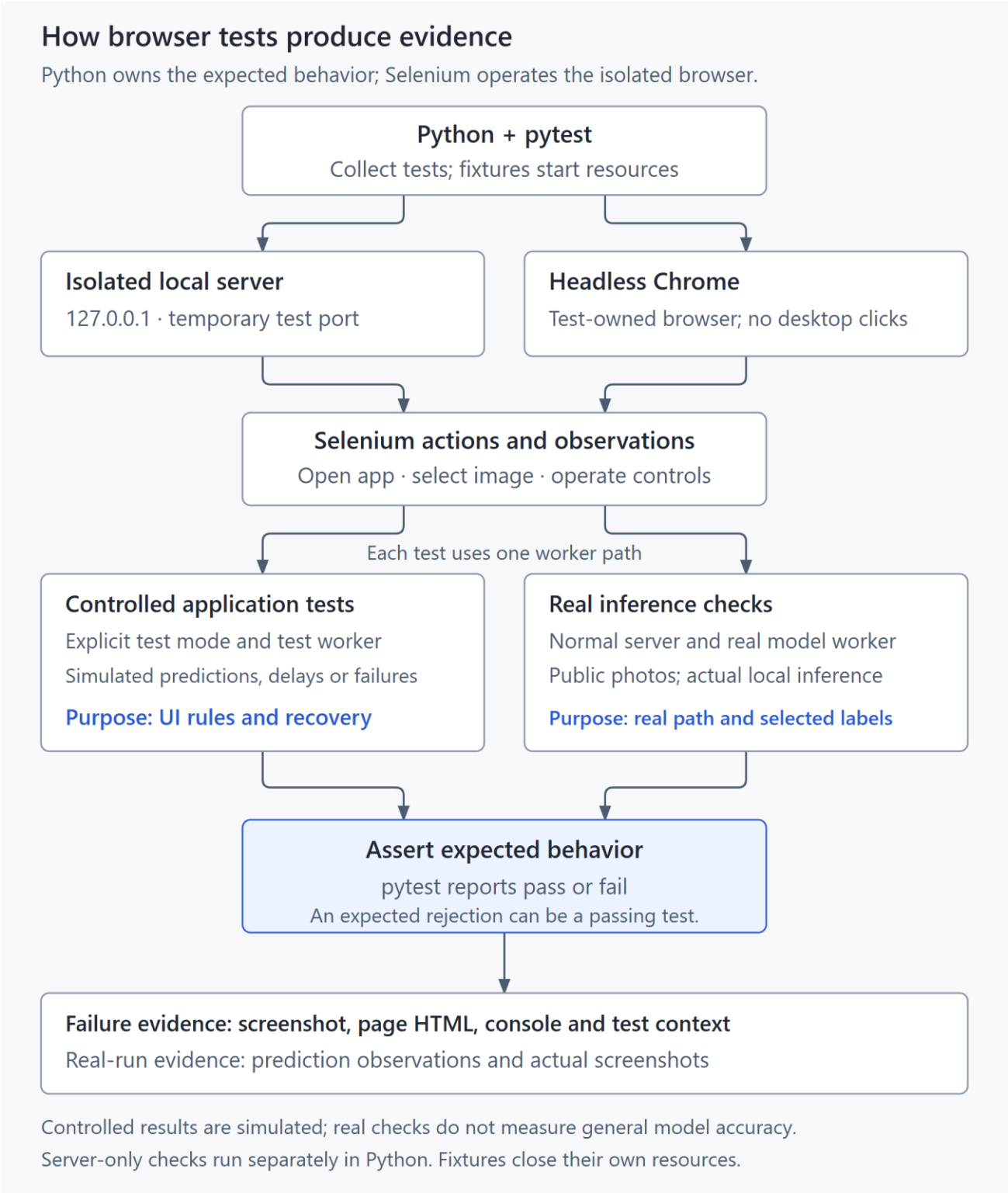


Figure 5 pytest runs written checks through Selenium. Controlled worker responses make interface scenarios repeatable; separately marked checks exercise the actual local model.

A worked requirement

The maximum file size is inclusive: a valid file exactly 10 MiB must be accepted, while a file one byte larger must be rejected. This is a boundary test because defects often occur at the point where accepted input becomes rejected input. A test far below the limit would not establish the inclusive rule.

Excerpt 2 Inclusive byte-size boundary and useful error content

```
def test_file_size_boundary(app, make_image, size, expected_state):
    upload(app, make_image("size-boundary.png", total_bytes=size))
    wait_state(app, expected_state, timeout=20)
    if expected_state == "error":
        assert_actionable_error(app, "10 mib", "smaller", "jpeg", "png")
        assert result_labels(app) == []
    else:
        assert result_labels(app) == ["laptop", "cup"]
```

Source: tests/test_ui.py: test_file_size_boundary

In tests/test_ui.py, the parameterization above this function runs the same test at two sizes: exactly 10 MiB and one byte above it. Read the function in four parts: prepare a valid image with the chosen byte count; submit it through Selenium; wait for an observable state; assert the expected rejection or controlled prediction list. The image must remain decodable so the case reaches the intended size rule. The laptop/cup labels are deliberate test-worker responses, not recognition results.

A negative test can pass when the application displays an error. For an oversized file, rejection with a useful message is the required behavior. pytest reports failure when the observed behavior differs from that requirement; it does not independently decide what the product ought to do.

Back to contents

Requirements and test design

Acceptance criteria describe externally observable behavior. The test plan chooses inputs and actions that could reveal a violation. Mapping a test to a requirement makes a failure interpretable and exposes requirements that have only manual or limited evidence.

Table 5 Acceptance criteria and check locations

ID	Acceptance criterion	Representative checks
AC1	Accept valid JPEG/PNG within byte and pixel limits; reject invalid input usefully	test_ui.py: test_file_size_boundary, pixel boundary, invalid/truncated content and JPEG orientation
AC2	The current image has one matching row for each displayed box	test_ui.py: visible names/scores, identity and geometry; test_inference.py: three selected-box regressions
AC3	Selection highlights its matching box; Clear removes current content	test_ui.py: result selection, replacement and keyboard Clear
AC4	Older results cannot repaint a replacement image or cleared view	test_ui.py: delayed result replacement and Clear; test_sample.py: late fetch after Clear/upload
AC5	Invalid input, empty results and inference failure remain distinct and recoverable	test_recovery.py: malformed output, loading/inference failure, Clear and successful retry
AC6	Keyboard controls, narrow layout and configured status/error announcements remain usable	test_ui.py: viewport, visible error content, roles and live-region settings; keyboard checks

ID	Acceptance criterion	Representative checks
AC7	The filter consistently selects retained current predictions using raw scores	test_filter.py: raw-score boundary, restore without inference, duplicate/stable identity, selection, reset, errors and keyboard
AC8	Try sample follows ordinary image validation and inference	test_sample.py: fetch/decode failure, timeout, replacement and keyboard; test_inference.py: real coffee through Try sample
AC9	PNG output contains the current visible annotations at oriented resolution	test_export.py: orientation, retained/excluded pixels, failed/late encoding and recovery
AC10	Startup is bounded and preserves the correct current state	test_startup.py and test_loading.py: configuration deadline, preparation, progress and retired workers
AC11	The package exposes only approved public files and runs with an ordinary static server	test_public_package.py: allowlist, hashes, privacy/provenance and actual model/export

Source: tests/test_ui.py; tests/test_filter.py; tests/test_sample.py; tests/test_export.py; tests/test_inference.py; tests/test_recovery.py; tests/test_startup.py; tests/test_loading.py; tests/test_public_package.py

The check locations identify assertions in the documented source. The execution record in Verification and release assessment identifies which checks actually ran. Controlled fixture passes establish the specified application behavior, while actual-model runs supply a different and limited form of evidence.

Choosing useful cases

The score-filter checks include 0.7999, exactly 0.80 and 0.95 against a minimum of 0.80. The first prediction is excluded even though its displayed rounded score is 0.80; the other two remain. A message counter verifies that filtering and restoring predictions do not trigger a second inference. Separate cases check stable identity, color, selection, an all-hidden state and new-image reset.

The PNG checks verify decoded dimensions, including a JPEG stored at 320 by 240 pixels that displays and exports at 240 by 320 after EXIF rotation. Known pixel locations check for a retained box, an excluded box and an unchanged background. Encoder-failure and delayed-callback cases test recovery and cancellation. The actual-model coffee export separately demonstrates the full integration.

Source: tests/test_export.py

Exploratory testing complements the scripted cases. A short charter is to change input while the app is busy, adjust the filter with the keyboard and inspect whether every visible result still belongs to the displayed image. Record the inputs and observations so an unexpected result can be reproduced.

A long repeated-use session checks a different risk from a single functional test. It can reveal observed state or recovery failures during that session, but it does not certify memory-leak freedom, production availability or concurrent-user scalability.

Status, input errors, loading feedback and export feedback have automated checks for useful visible content and the intended announcement configuration. The export-message element remains present as a polite live region even when empty. These checks inspect page content and DOM settings; they do not certify actual screen-reader speech or complete accessibility.

Source: dist/index.html; dist/style.css

Visible results and box regression

Reading only data attributes can miss an empty visible label. The strengthened checks inspect rendered names in result rows and corresponding box labels, and formatted scores in result rows. Deliberately blanking that text in a disposable copy must fail the assertions even if data-label and data-score remain correct.

Source: tests/test_ui.py; tests/test_inference.py; tools/check_review_sensitivity.py

The three reference boxes come from the earlier output of the same pinned model in model-observations.json. Each normalized edge may differ by 0.005, which is 0.5 percent of the image extent and about 2 to 3 source pixels for these samples. Rendered positions allow that scaled edge tolerance plus 2 CSS pixels for layout rounding/borders; widths and heights allow two edge tolerances plus 2 CSS pixels. Actual rendered position and size are measured relative to the displayed image. Deliberately displacing every box must fail even though its label and count remain unchanged.

Source: tests/test_inference.py; docs/evidence/model-observations.json

These comparisons answer whether a selected result or its rendering changed. Independent human box annotations would be needed to assess localization accuracy. Neither a good match nor a high score establishes that all predictions are correct.

Back to contents

Failure investigation

A useful failure report states the expected rule, the exact steps and input, the observed behavior and the evidence needed to reproduce it. Start with what happened before proposing a cause. A screenshot shows the visible symptom; browser logs, test assertions and request identities can explain the path that produced it.

Table 6 Evidence to retain

Evidence	Question it helps answer
Test name, assertion and command	Which expected rule disagreed with the observed state
Input and controlled scenario	Whether another run can repeat the same conditions
Screenshot and page HTML	What the page actually displayed at the failure
Browser console output	Whether runtime or worker errors accompanied the symptom
Source version and environment	Which implementation and browser/runtime were exercised
Follow-up result	Whether the proposed fix addressed the original reproduction

A real narrow viewport defect

The extension review found a genuine layout defect at a 320 CSS-pixel viewport. A vertical scrollbar left 305 pixels available to the document, but the body still had a 320-pixel minimum width. The page overflowed horizontally. The preserved failing test reports the comparison 320 <= 305 + 1 as false.

Table 7 Narrow viewport before and after the correction

Measurement	Before	After
Requested CSS viewport	320 px	320 px
Available document width	305 px	305 px
Document content width	320 px	305 px
Overflow beyond available width	15 px	0 px

The earlier check compared content width with window.innerWidth, which included space unavailable to the content when the scrollbar was present. It could therefore pass while the page still overflowed. The corrected check sets the CSS viewport explicitly and compares scrollWidth with documentElement.clientWidth, allowing one pixel for rounding. It also checks that each essential control fits within the available width.

Changing the body's min-width from 320px to 0 lets the responsive layout use the actual available width. A subsequent real-model capture measured content and available widths of 305 and 305 at 320, and 375 and 375 at 390. The 768- and 1440-pixel layouts also fit their available widths. This finding combines an application fix with a stronger test measurement; it was not an injected source fault.

Source: docs/evidence/viewport-before-fix.xml; dist/style.css; tests/test_ui.py; docs/evidence/extension-demo.json

A deliberate stale-result fault

The base verification included an isolated experiment that disabled the current-image revision guard. A delayed result for an earlier image could then appear on a replacement image, and the corresponding test detected it. This demonstrates that the check can expose that chosen fault. The fault was deliberately introduced in an isolated copy; it was not a naturally discovered production incident.

The investigation connects the symptom to a narrow mechanism: an old request returned after the displayed image changed, and the altered copy no longer rejected it. Restoring the guard addresses that mechanism. This is first-level application investigation, not a claim about a defect inside the pretrained model.

A prediction mistake is a different kind of finding

The real base coffee run included duplicate spoon predictions, while another sample produced incorrect and duplicate labels. Those observations describe model behavior. If the application accurately renders the returned data, an incorrect model label is not by itself proof that the rendering code is broken.

The response should match the finding: fix a reproducible application defect, improve the wording if the UI overstates a prediction, or record a model limitation. Raising a score filter may hide some predictions but does not establish general accuracy improvement.

An unexpected file in a download check

The first combined extension run recorded 47 passes and one failure. The failure occurred when a timeout test expected an empty download folder but found downloads.htm.crdownload. The retained investigation identified a non-PNG CRX3 container holding model data. The producing process or component was not established, so the finding does not prove that Object Lens generated that file.

Source: docs/evidence/extension-first-full-suite.xml; docs/evidence/extension-verification.json

The strict assertion was retained. The isolated test browser was then configured to resolve only localhost and 127.0.0.1 because its application and model assets are served locally. This change is confined to the test browser and does not alter the computer's network or security settings. The subsequent complete rerun passed all 48 cases.

Source: tests/conftest.py; docs/evidence/extension-full-suite.xml

A slow connection that could never finish

An independent audit paced only the 43 MB model response at 2.5 Mbps on loopback. The fixed 120-second loading deadline fired with 86 percent received, and Try again restarted the transfer from zero. A steady connection below about 2.9 Mbps could therefore never load the detector, although the public-demo note said slow connections could recover by retrying.

Source: docs/SOURCE_REVIEW.md; docs/evidence/loading-followup-slow-transfer.json

The deadline was protecting against a dead connection, not a slow one, so it now measures a stall. Each loading event restarts a 45-second window, and runtime initialization after the model file finishes keeps its own 120-second limit. Controlled tests cover a stalled transfer, a slow steady transfer and an initialization hang. The same paced transfer then completed and the detector became ready at 139.6 seconds. This was an observed defect, not an injected fault.

Source: dist/app.js; tests/test_recovery.py; docs/evidence/loading-followup-slow-transfer.json

Current recovery wording and assertion gaps

The preceding timeout callback used an image-detection explanation while the model was still loading. Clear then described any failed worker as a startup failure. The follow-up separates the failure stages and preserves a truthful unavailable state until retry succeeds. Controlled reproductions check the visible explanation, absence of old results and subsequent recovery.

Source: docs/SOURCE_REVIEW.md; tests/test_recovery.py

Fresh disposable-copy experiments validate the strengthened assertions. A shared 11-case clean baseline passes first, followed by nine deliberate-fault copies: blank names or scores, displaced boxes, removed announcement settings, a fixed load deadline, an extended initialization deadline, accepted blank labels and consistently scaled scores. The records retain each source change and its intended failing assertion. A deliberate source fault demonstrates sensitivity to that specific mistake; it is not a production incident or a broad mutation-testing score.

Source: docs/evidence/loading-followup-sensitivity.json

Back to contents

Verification and release assessment

The current loading follow-up was executed on 3 November 2026. Its exact commands, environment, source hashes and selected evidence are recorded in loading-followup-verification.json. The starting commit is 17f2b9a; the recorded file hashes identify the tested changes. Checkpoints overlap and are never added together.

Table 8 Current and historical execution records

Check	Observed result	Evidence and limit
Current combined suite	98 passed; 0 failures/errors/skips; 783.414 s JUnit	94 non-model and 4 actual-model cases; loading-followup-full-suite.xml
Current deliberate faults	11 clean passes; 15 intended failures across 9 faults	loading-followup-sensitivity.json; selected faults in disposable copies
Current rendered review	12 captures inspected; 320/390/768/1440 px layouts fit	loading-followup-rendered-review.json; real coffee workflow and labelled controlled recovery capture
Current static package	23 entries verified; 34,572,573 bytes in ZIP	loading-followup-package.json; prepared locally, not deployed
Slow-transfer probe	2.5 Mbps: ready at 139.6 s; previous source failed at 121 s	loading-followup-slow-transfer.json; loopback pacing, one run each
Preceding review follow-up	93 passed; 648.509 s	Historical 3 November source; followup-full-suite.xml
Preceding correctness review	72 passed; 531.53 s	Historical 3 November source; review-full-suite.xml
Public preparation	59 passed; 441.33 s	Historical 3 November preparation; public-prep-full-suite.xml
Combined extension	48 passed; 418.62 s	Historical 2 November extension; extension-full-suite.xml
Base repeated use	320 cycles; 3615.927 s	Historical base only; no current flake-rate or memory-leak claim

The before/after recovery comparison ran the same three controlled wording checks against the 74d108f source and the current source: three expected pre-Clear failures in 27.061 seconds, followed by three passes in 28.254 seconds. Only the passing current checks continue through Clear, unavailable feedback, Retry and a valid new image. The baseline Clear behavior is not independently reached by those failed checks.

Source: docs/evidence/loading-followup-recovery-comparison.json

The recorded current environment is Python 3.12.10, pytest 9.0.3, Selenium 4.40.0, Chrome 156.0.8078.80 and ChromeDriver 156.0.8078.80. This was one Windows laptop with headless Chrome and prepared local assets. Workstation load was not controlled, so timing is an observation rather than a performance promise.

Source and evidence hashes bind observations to inspectable bytes. Selected JUnit copies remove the machine hostname; the raw hashes and redaction are retained in the manifest. New scripts for sensitivity, before/after recovery comparison, rendered capture and verification recording are tracked under tools. Historical helpers that existed only in ignored artifacts are not claimed to have been reconstructed.

The three real-photo box references are earlier pinned-model outputs, not independent annotations. Scores remain model scores, duplicate predictions are still possible and the filter only changes the displayed set. Malformed-response and timeout cases use deliberate controlled failures. The slow-

transfer probe paced one response on loopback; it is not an Internet speed measurement. Announcement checks establish configured DOM behavior, not actual screen-reader output.

Release decision

The current combined result is 98 passed, 0 failures/errors/skips; 783.414 seconds of JUnit suite time (94 non-model and 4 real-model cases). A paced 2.5 Mbps model transfer that the previous source could not complete now reaches ready. Together with the targeted failure/recovery checks, deliberate-fault sensitivity and current rendered review, it supports the agreed local learning and demonstration workflow. The refreshed book and static ZIP are local deliverables. Website upload remains parked and no public URL has been verified.

Other browsers/devices, Safari/iOS, hosted CI, public deployment, a general accuracy dataset, GPU/NPU execution, semiconductor inspection and production scale remain outside the evidence. The old one-hour session does not requalify the current revision. Retry still restarts the model download from the beginning. Repository content also cannot prove the owner's personal understanding or unaided authorship.

Back to contents

Running and maintaining the project

Use the application

Start the local server

```
.\.venv\Scripts\python.exe serve.py
```

Open http://127.0.0.1:8765 in Chrome and keep the terminal running. Ctrl+C stops the server. If that port is already in use, choose another with serve.py --port 8766 and open the corresponding local address.

For a short demonstration, use Try sample, wait for the real predictions, select a result, raise and lower the minimum score, then save the visible annotations. Open the resulting PNG separately and compare it with the current view. Finally choose a different photo or Clear.

Run checks

Execute the suite

```
.\.venv\Scripts\python.exe -m pytest
```

The tests own their temporary servers and headless browsers. They should clean up those resources when finished. A manually opened app can remain available in another terminal; do not confuse its state with the isolated test browser.

Table 9 Where to look

Location	Purpose
dist	The application and prepared local runtime/model/sample assets
tests	Python acceptance checks, server checks and browser fixtures
docs/DESIGN.md	Concise design and acceptance criteria

Location	Purpose
docs/VERIFICATION.md and docs/evidence	Recorded results and selected evidence
tools and asset-lock.json	Preparation helpers and exact asset sources/hashes
docs/system-design/source.json	Authoritative prose, tables and captions for this companion
docs/system-design/diagrams	Editable diagrams and their SVG/PNG exports
.venv, .local and artifacts	Generated environments and local working or raw-evidence files

Make a bounded change

Change one behavior with its acceptance criterion and relevant tests. Run the focused checks while developing, then the applicable wider suite and UI review before committing. A test that simply repeats the implementation's calculation can reproduce the same defect, so expected outcomes should follow the requirement or an independent example.

Keep the app's image identity, visible predictions and exported output consistent when changing result behavior. Keep real inference separate from controlled fixtures. Preserve existing attribution and the recorded asset sources when updating dependencies.

Update source.json and the relevant diagram source when the implementation changes, then rebuild the Word document and its matching PDF. The contents use internal hyperlinks and page numbers verified against the rendered document. Recheck those targets and every final page after changes. Editing a generated Word or PDF copy alone does not update the authoritative source.

AI assistance and honest demonstration

AI tools assisted with implementation, test authoring and documentation. The app adapts an upstream object-detection example and uses an existing pretrained model. It does not claim that this project trained the detector or that every automated check was independently written by the owner.

A demonstration can honestly describe the application workflow, the role of each technology, one acceptance criterion and its test, an investigated failure, and the scope of the release decision. The owner's understanding should be demonstrated through explanations and actions actually performed, without inventing prior experience.

The current public package has an exact allowlist: application files, normal-mode configuration, six pinned runtime/model assets, the public coffee sample and license/provenance texts. Tests, documents, environments and Git history are excluded. An ordinary static-server integration test verifies the package locally. Source publication through GitHub Desktop and website deployment remain separate owner actions.

Source: tools/build_public_demo.py; tests/test_public_package.py; docs/PUBLIC_DEMO.md

Back to contents

Terms and source references

Table 10 Terms in this application

Term	Meaning
Inference	Running a trained model on an input to produce predictions
Pretrained model	A model whose learned parameters were produced before this application used it
Model score	The number the model assigns to a prediction; not a measured accuracy percentage
Threshold	A selected minimum used to include or exclude predictions
Bounding box	A rectangle describing the predicted location of an object
WebAssembly	A browser execution format used by this runtime to run model operations on the CPU
Quantization	Using a lower-precision model representation; this application selects a prepared quantized model instead of training one
Web Worker	A browser execution context separate from the page's main thread
Fixture	Reusable test setup, inputs or controlled conditions
Assertion	A check comparing an observation with an expected rule
Regression	Previously correct behavior becoming incorrect after a change
Stale response	A result belonging to an earlier image or operation
Acceptance criterion	A concrete observable condition the implemented feature must satisfy
Release decision	A conclusion about readiness for a stated use, based on evidence and known limits

Application behavior is established by the documented source and its retained execution records. Upstream documentation explains library concepts and APIs; it is not evidence that this application passed a check.

Table 11 Source index

Subject	Source
Approved scope and acceptance criteria	docs/DESIGN.md
User workflow and commands	README.md and docs/LEARNING.md
Application state and rendering	dist/app.js, dist/png-export.js, dist/index.html and dist/style.css
Actual model execution	dist/worker.js and asset-lock.json
Test setup and behavior	tests/conftest.py; UI, recovery, inference, startup, loading, filter, sample, export, server and public-package tests
Verification records	docs/VERIFICATION.md; loading-followup-verification.json; loading-followup-sensitivity.json; historical records remain dated

Subject	Source
Reproducible current evidence	tools/check_review_sensitivity.py, check_recovery_comparison.py, capture_review.py and record_verification.py
Public sample provenance	docs/IMAGE_SOURCES.md
Upstream example and attribution	NOTICE.md and baseline/source.json
Transformers.js pipeline documentation	Transformers.js 2.17.2 pipeline API
Selenium documentation	Selenium browser automation
pytest documentation	pytest test runner

Back to contents